

LAPORAN PRAKTIKUM JOBSHEET 15



Disusun Oleh:

HAFIZH ARRASYIID SYAHBANA

254107060026

SIB 1A

**PROGRAM STUDI D4 SISTEM INFORMASI
BISNIS JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI
MALANG 2026**

13.2 Persiapan

 Customer10.java src 1

```
1  public class Customer10 {
2      public int id;
3      public String name;
4
5      public Customer10() {
6      }
7
8      public Customer10(int id, String name) {
9          this.id = id;
10         this.name = name;
11     }
12
13     public String toString() {      Add @Override Annotation
14         return " ID: " + this.id + " Nama: " + this.name;
15     }
16 }
```

 Book10.java src 1

```
1  public class Book10 {
2      public String isbn;
3      public String title;
4
5      public Book10() {
6      }
7
8
9      public Book10(String isbn, String title) {
10         this.isbn = isbn;
11         this.title = title;
12     }
13
14     public String toString() {      Add @Override Annotation
15         return " ISBN: " + this.isbn + " Title: " + this.title;
16     }
17 }
```

13.3 Praktikum – Implementasi ArrayList

ArrayList merupakan sebuah class dari Java Collection Framework yang menyediakan implementasi struktur data serupa dengan array tetapi dengan ukuran dinamis.

1. Buat file DemoArrayList.java. Lakukan import java.util.ArrayList;

```
📄 DemoArrayList10.java src
1  import java.util.ArrayList;
2
3  public class DemoArrayList10 {
```

2. Pada fungsi main(), instansiasi collection baru dengan nama customers bertipe ArrayList of Customer dengan size 2. Selanjutnya, buat object customer1 dan customer2 kemudian tambahkan ke dalam ArrayList customers dengan method add.

```
ArrayList<Customer10> customers = new ArrayList<>(initialCapacity: 2);

Customer10 customer1 = new Customer10(id: 1, name: "Zakia");    "Zakia"
Customer10 customer2 = new Customer10(id: 5, name: "Budi");    "Budi": 1

customers.add(customer1);
customers.add(customer2);
```

3. Gunakan looping dengan foreach untuk mencetak data customers

```
for (Customer10 cust : customers) {    "
    System.out.println(cust.toString());
}
```

4. Cobalah tambahkan object customer baru ke dalam customers. Apakah object dapat ditambahkan meskipun melebihi kapasitas?

```
customers.add(new Customer10(id: 4, name: "Cica"));
```

Ya, object tetap dapat ditambahkan. ArrayList bersifat dinamis dan akan otomatis memperbesar kapasitasnya di belakang layar meskipun size inisial (misalnya 2) telah terlampaui.

5. Compile dan run kode program, di mana object yang baru ditambahkan? Di awal, di tengah, atau di akhir collection?

Secara default, method .add(Object) akan menambahkan elemen baru di akhir collection.

6. Untuk menambahkan object baru pada index tertentu, lakukan sebagai berikut

```
customers.add(index: 2, new Customer10(id: 100, name: "Rosa"));
```

7. Compile dan run kode program. Index pada ArrayList dimulai dari 0 atau 1?

Index pada ArrayList dimulai dari 0.

8. Untuk mengetahui posisi dari suatu objek, gunakan method indexOf()

```
System.out.println(customers.indexOf(customer2));
```

9. Untuk mengembalikan object pada index tertentu, gunakan method get()

```
Customer10 customer = customers.get(index: 1);  
System.out.println(customer.name);  
customer.name = "Budi Utomo";    "Budi": Unknown word
```

10. Cobalah hapus angka 2 saat instansiasi object customers. Apakah ArrayList dapat diinstansiasi tanpa harus menentukan size di awal?

Ya, ArrayList dapat diinstansiasi tanpa menentukan ukuran awal. Java akan secara otomatis memberikan kapasitas default (biasanya berukuran 10).

11. Anda juga dapat menambahkan sekumpulan customer baru ke dalam ArrayList secara sekaligus. Misalnya terdapat ArrayList newCustomers. Tambahkan seluruh object customer sekaligus ke dalam customers.

```
ArrayList<Customer10> newCustomers = new ArrayList<>();  
newCustomers.add(new Customer10(id: 201, name: "Della"));  
newCustomers.add(new Customer10(id: 202, name: "Victor"));  
newCustomers.add(new Customer10(id: 203, name: "Sarah"));  
  
customers.addAll(newCustomers);  
  
for (Customer10 cust : customers) {    "cust": Unknown word  
    System.out.println(cust.toString());    "cust": Unknown word  
}
```

12. Karena sudah menyediakan method toString(), pengecekan data customers untuk proses debugging juga dapat dilakukan lebih sederhana dengan cara berikut

```
System.out.println(customers);
```

13.4 Praktikum – Implementasi Stack

Class Stack dalam Java Collection Framework yang menyediakan implementasi struktur data tumpukan/stack.

1. Buat file StackDemo.java. Lakukan import java.util.Stack;

```
StackDemo10.java src
1  import java.util.Stack;
2
3  public class StackDemo10 {
```

2. Pada fungsi main() buat beberapa object bertipe Book.

```
Book10 book1 = new Book10(isbn: "1234", title: "Dasar Pemrograman");
Book10 book2 = new Book10(isbn: "7145", title: "Hafalah Shalat Delisa");
Book10 book3 = new Book10(isbn: "3562", title: "Muhammad Al-Fatih");
```

3. Instansiasi object books bertipe Stack of Book kemudian tambahkan object yang sudah dibuat ke dalamnya menggunakan method push()

```
Stack<Book10> books = new Stack<>();
books.push(book1);
books.push(book2);
books.push(book3);
```

4. Class Stack juga sudah memiliki method pop() dan peek() seperti yang Anda diimplementasikan secara manual pada praktikum sebelumnya

```
Book10 temp = books.peek();
if (temp != null) {
    System.out.println(temp.toString());
}

Book10 temp2 = books.pop();
if (temp2 != null) {
    System.out.println(temp2.toString());
}
```

5. Mengapa perlu ada pengecekan (temp != null)?

Pengecekan ini dilakukan sebagai bentuk defensive programming untuk menghindari error NullPointerException. Jika objek temp ternyata bernilai null (kosong), lalu program mencoba memanggil method pada objek tersebut (seperti temp.toString()), maka program akan crash atau error. Dengan menambahkan pengecekan (temp != null), kita memastikan bahwa perintah mencetak di layar hanya akan dieksekusi jika objek buku tersebut benar-benar ada datanya.

6. Lakukan looping untuk mencetak data buku pada stack

```
for (Book10 book : books) {  
    System.out.println(book.toString());  
}
```

7. Jika diperlukan pada proses debugging, books juga dapat dicetak dengan cara berikut

```
System.out.println(books);
```

8. Bagaimana cara melakukan pencarian elemen pada stack menggunakan method search()?

Method search(Object o) pada java.util.Stack digunakan untuk mencari seberapa jauh sebuah objek berada dari tumpukan paling atas (top of stack).

- Cara kerja indeks: Posisi dihitung mulai dari angka 1 (bukan 0). Elemen yang paling terakhir dimasukkan (paling atas) berada di posisi 1, elemen di bawahnya posisi 2, dan seterusnya.
- Jika tidak ditemukan: Jika objek yang dicari tidak ada di dalam stack, method ini akan mengembalikan nilai -1.

13.5 Praktikum – Implementasi TreeSet

Class TreeSet pada Java Collection Framework menyediakan implementasi binary search tree. Lakukan langkah percobaan berikut:

1. Buat file TreeSetDemo.java kemudian import java.util.TreeSet;

```
TreeSetDemo10.java src  
1  import java.util.TreeSet;  
2  
3  public class TreeSetDemo10 {
```

2. Tambahkan fungsi main() kemudian instansiasi object TreeSet of String.

```
TreeSet<String> fruits = new TreeSet<>();

fruits.add(e: "Mangga");    "Mangga": Unknown word.
fruits.add(e: "Apel");      "Apel": Unknown word.
fruits.add(e: "Jeruk");     "Jeruk": Unknown word.
fruits.add(e: "Jambu");     "Jambu": Unknown word.

for (String temp : fruits) {
    System.out.println(temp);
}
```

3. Tambahkan beberapa nilai bertipe String ke dalam TreeSet

```
for (String temp : fruits) {
    System.out.println(temp);
}
```

4. Compile dan run program. Mengapa urutan yang ditampilkan berbeda dengan urutan penambahan data ke dalam TreeSet fruits?

Urutannya berbeda karena sifat utama dari TreeSet di Java adalah menyimpan data secara terurut (sorted) berdasarkan natural ordering (urutan alami). Karena tipe data yang digunakan adalah String, maka TreeSet akan secara otomatis mengurutkan data tersebut sesuai abjad (A-Z). Meskipun ditambahkan secara acak (Mangga, Apel, Jeruk, Jambu), saat dicetak hasilnya akan menjadi: Apel, Jambu, Jeruk, Mangga. TreeSet menggunakan struktur data Tree (biasanya Red-Black Tree di bawah kapnya) yang secara otomatis menyusun posisi setiap node agar selalu terurut.

5. Tambahkan kode program sebagai berikut:

```
System.out.println("First: " + fruits.first());
System.out.println("Last: " + fruits.last());

fruits.remove(o: "Jeruk");    "Jeruk": Unknown word.
System.out.println("Setelah remove " + fruits);

fruits.pollFirst();
System.out.println("Setelah poll First " + fruits);

fruits.pollLast();
System.out.println("Setelah poll Last " + fruits);
```

6. Apa yang dilakukan oleh method `first()`, `last()`, `remove()`, `pollFirst()`, dan `pollLast()`?

`first()`: Method ini berfungsi untuk mengembalikan (mengambil nilainya saja tanpa menghapus) elemen pertama (paling kecil atau secara alfabet paling awal) yang ada di dalam set saat itu.

`last()`: Method ini berfungsi untuk mengembalikan elemen terakhir (paling besar atau secara alfabet paling akhir) yang ada di dalam set saat itu.

`remove(Object o)`: Method ini berfungsi untuk menghapus elemen spesifik yang disebutkan di dalam kurung (parameter) dari set. Jika elemen tersebut berhasil dihapus, ukurannya berkurang.

`pollFirst()`: Method ini berfungsi untuk mengambil DAN menghapus elemen pertama (paling kecil/awal) dari set. Jika set kosong, method ini akan mengembalikan nilai null.

`pollLast()`: Method ini berfungsi untuk mengambil DAN menghapus elemen terakhir (paling besar/akhir) dari set. Jika set kosong, method ini juga akan mengembalikan nilai null.

13.6 Praktikum – Sorting

Selain pencarian data menggunakan method `get()` atau `search()`, Java Collection Framework juga menyediakan fungsi untuk melakukan pengurutan data.

Untuk melakukan pengurutan data String, int, atau primitive data type lain, Anda dapat melakukan cara berikut:

```
ArrayList<String> daftarSiswa = new ArrayList<>();  
daftarSiswa.add(e: "Zainab");  
daftarSiswa.add(e: "Andi");  
daftarSiswa.add(e: "Rara");  
Collections.sort(daftarSiswa);  
  
System.out.println(daftarSiswa);
```

Khusus untuk collection of objects, pengurutan data harus menentukan berdasarkan atribut mana pengurutan dilakukan. Misalnya ingin dilakukan pengurutan data customer pada praktikum di atas berdasarkan atribut name. Gunakan kode program berikut (berikan nama variabel `c1` dan `c2` secara random. Anda bisa menggunakan nama variabel lainnya)


```
customers.sort((c1, c2) -> c1.name.compareTo(c2.name));  
System.out.println(customers);
```

Kedua cara di atas dapat digunakan untuk mengurutkan data pada jenis collection lain yang tidak melakukan pengurutan secara otomatis seperti TreeSet.

LINK GITHUB : <https://github.com/nNasq/Jobsheet-15---Collection>